Subexponential Set Splitting from Exact SAT Counting

GPT-6
OpenAI Codex

29 October 2026

Abstract

We reduce Set Splitting to exact counting for 3-CNF formulas. A hypergraph with n vertices and m edges needs $m+1$ counting calls, each with $O(n+m)$ variables, regardless of edge size. The counts determine how many colorings split each possible number of edges. Using the general counting bound stated in the October 2026 SAT manuscript, this gives a deterministic polynomial-space algorithm with running time

$$|I|^{O(1)} + 2^{O(k \log\log k/\sqrt{\log k})},$$

where k is the number of edges to split. We state separately the weaker bound supported by its formalization scope note. The reduction is proved here; the SAT counting result is assumed.

1 Problem and assumptions

Set Splitting asks whether a two-coloring can make at least k sets contain both colors. Sets may have any size. Lokshtanov and Saurabh [1] give an equivalent instance with at most k vertices and $2k$ edges. Earlier specialized algorithms take $O^*(1.7171^k)$ time with exponential space or $O^*(1.8213^k)$ time with polynomial space [2]. Here O^* hides polynomial factors.

Let $H = (V, (e_i)_{i=1}^m)$ be a hypergraph with $V = [n]$. Edges have separate labels, so repeated edges count separately. Write

$$s = n + m, \qquad J = \sum_i |e_i|.$$

The vertices and edges are listed explicitly; L is the input bit length. Empty and singleton edges are never split. For this paper, both count as monochromatic.

For $\alpha \in \{1, 2\}$, define

$$f_\alpha(v) = \frac{v(\log\log v)^\alpha}{\sqrt{\log v}}.$$

Small arguments are handled directly. We use the following external assumption.

Definition 1.1 (SAT counting assumption)**.** Count_α is a uniform deterministic algorithm that counts the satisfying assignments of a 3-CNF with v declared variables and encoding length ℓ in time

$$(\ell+1)^{O(1)} 2^{O(f_\alpha(v))}$$

and polynomial space.

The revised SAT manuscript states Count_1 [3]. Its formalization scope note documents Count_2 [4]. We keep these rates separate and do not reprove either result. All formulas below have width at most three. Unused variables remain declared and contribute their usual factor of two.

2 Counting selected monochromatic edges

For a coloring $x \in \{0,1\}^n$, let $u(x)$ be the number of monochromatic edges. Define

$$a_j = |\{x : u(x) = j\}|, \qquad P_H(z) = \sum_{j=0}^{m} a_j z^{m-j}.$$

Thus the coefficient of z^b counts colorings that split exactly b edges. This is a form of the hypergraph coboundary polynomial [5].

We first compute

$$F_t = \sum_{x \in \{0,1\}^n} \binom{u(x)}{t}, \qquad 0 \le t \le m. \tag{1}$$

These counts select t monochromatic edges, then sum over all colorings.

Lemma 2.1 (Recovering the distribution)**.** *The counts in (1) give*

$$a_j = \sum_{t=j}^{m} (-1)^{t-j} \binom{t}{j} F_t. \tag{2}$$

This calculation takes polynomial time and space.

Proof. The binomial theorem gives

$$\sum_{t=0}^{m} F_t w^t = \sum_{x} (1+w)^{u(x)} = \sum_{j=0}^{m} a_j (1+w)^j.$$

Replace w by $z-1$ and compare coefficients. Each $F_t \le 2^{n+m}$. Each term in (2) has magnitude at most 2^{n+2m}, so partial sums need only $O(n+m+\log(m+1))$ bits. There are $O(m^2)$ terms. □

Lemma 2.2 (Counting selected bits)**.** *The condition that exactly t of m bits equal one has a width-three CNF encoding with $O(m+1)$ extra variables and clauses. Each valid input assignment has exactly one extension to the extra variables.*

Proof. Sum the bits using a balanced tree of binary adders, then fix the sum to t. At level h, there are $O(m/2^h)$ additions of $O(h)$-bit numbers. The gate count is

$$O\left(m \sum_{h \ge 1} \frac{h}{2^h}\right) = O(m).$$

Padding with zeros handles other input lengths. Encode each gate by its full logical equivalence. For example, $g \leftrightarrow (a \wedge b)$ needs

$$(\neg g \vee a) \wedge (\neg g \vee b) \wedge (g \vee \neg a \vee \neg b).$$

OR, NOT, and XOR also have constant-size width-three encodings. The inputs determine every gate value, giving the required unique extension. The case $m = 0$ takes constant space. □

Theorem 2.3 (SAT reduction)**.** *Each F_t is the satisfying-assignment count of a 3-CNF with $O(n+m+1)$ variables and $O(J+m+1)$ clauses. The formula can be built in polynomial time.*

Proof. Use color variables $x_1, \dots, x_n$ and one selector y_i per edge. For each nonempty edge e_i, choose an anchor $a_i \in e_i$. For every $v \in e_i \setminus \{a_i\}$, add

$$(\neg y_i \vee \neg x_{a_i} \vee x_v), \qquad (\neg y_i \vee x_{a_i} \vee \neg x_v). \tag{3}$$

These clauses say that selecting an edge forces all its vertices to have the same color. Empty and singleton edges need no clauses. Add the exact-t selector count from Lemma 2.2.

Fix a coloring x. Its allowed selector assignments are precisely the t-element subsets of its $u(x)$ monochromatic edges. There are $\binom{u(x)}{t}$ such assignments, each with one extension to the gate variables. Summing over x proves that the count is F_t.

The edge constraints use at most $2J$ clauses and no extra variables. The selector count adds $O(m+1)$ variables and clauses. The encoding length is $O((J+n+m+1)\log(n+m+2))$. □

The selectors use one-way implications: a monochromatic edge may be selected or left unselected. The gate constraints use equivalences so their extra variables do not multiply the count.

3 Running time

Theorem 3.1 (Complete distribution)**.** *Assume* Count_α*. All coefficients of* P_H *can be computed deterministically in polynomial space and time*

$$(L+1)^{O(1)} 2^{O(f_\alpha(n+m))}. \tag{4}$$

Proof. Build the formula in Theorem 2.3 for each $t = 0, \dots, m$ and count its satisfying assignments. Each formula has $v \le C(n+m+1)$ variables for an absolute constant C. Its length is polynomial in $L+1$, and $f_\alpha(C(s+1)) = O(f_\alpha(s))$. The $m+1$ calls and Lemma 2.1 add only polynomial work. Run the calls one at a time. Their outputs and all intermediate sums have polynomial bit length, so the total space is polynomial. □

Corollary 3.2 (Set Splitting)**.** *Under* Count_α*, Set Splitting with target* k *can be decided in polynomial space and time*

$$|I|^{O(1)} + 2^{O(f_\alpha(k))}.$$

Thus the revised SAT manuscript gives $2^{O(k \log\log k/\sqrt{\log k})} + |I|^{O(1)}$*. The scope note gives* $2^{O(k(\log\log k)^2/\sqrt{\log k})} + |I|^{O(1)}$*. Both bounds are subexponential in* k*.*

Proof. Apply the decision kernel of Lokshtanov and Saurabh [1, Theorem 1]. It answers the instance or returns an equivalent instance (H', k_0) with at most k vertices and $2k$ edges, where k is the original target. Use Theorem 3.1 on H'. Answer yes if a coefficient at an exponent at least k_0 is positive. The reduced input has polynomial length in $k+1$. Its polynomial factors are absorbed in the displayed exponential term. □

This kernel preserves the decision, not necessarily the number of colorings. The corollary counts the reduced instance. Theorem 3.1 counts the original instance in terms of $n+m$.

The SAT manuscript also states a faster bound $\mathrm{poly}(\ell+v)2^{O_D(v/\sqrt{\log v})}$ for formulas with at most Dv clauses, for fixed D. If $J = O(n+m)$, our formulas have fixed clause density, giving

$$(L+1)^{O(1)} 2^{O((n+m)/\sqrt{\log(n+m)})}.$$

This faster rate does not apply to every kernel: it may have $\Theta(k^2)$ incidences and only $O(k)$ variables.

4 Extensions

Fixed colors and an optimal coloring. Unit clauses fix vertex colors, so the same method counts their extensions. Let b be the largest exponent with a positive coefficient. Fix each vertex to zero if the coefficient at b stays positive; otherwise fix it to one. An optimum remains possible at every step. At most $n+1$ computations recover an optimal coloring within (4), up to a polynomial factor.

Families of 2-CNF formulas. Let $\Phi_1, \ldots, \Phi_m$ be 2-CNF formulas on the same n variables, with C clauses in total. For every clause D of Φ_i, add $\neg y_i \vee D$. Selecting i now forces Φ_i to hold. The exact-t selector count and binomial inversion recover the number of assignments making exactly j of the formulas true, for every j. Each SAT call uses $O(n+m+1)$ variables and $O(C+m+1)$ clauses, so Theorem 3.1 applies.

A clause is false exactly when all its literals are false, a conjunction of unit clauses. This also gives the satisfaction distribution for any CNF, regardless of clause width.

5 Checks and scope

Run `python3 research/verify_paper.py` from the repository root. The code checks the SAT counts against exhaustive assignments, including repeated edges, unused vertices, fixed colors, and the extensions above. Results and source hashes are in `paper/verification.json`.

The tests check the reduction, not the external SAT algorithm. The polynomial identity is established prior work [5]. The open question is whether arbitrary edge sizes admit the faster fixed-density bound with only $O(n+m)$ variables.

References

[1] Daniel Lokshtanov and Saket Saurabh. Even Faster Algorithm for Set Splitting! In *Parameterized and Exact Computation*, LNCS 5917, pp. 288–299, Springer, 2009. doi:10.1007/978-3-642-11269-0_24.

[2] Jesper Nederlof, Johan M. M. van Rooij, and Thomas C. van Dijk. Inclusion/Exclusion Meets Measure and Conquer. *Algorithmica*, 69(3):685–740, 2014. doi:10.1007/s00453-013-9759-2.

[3] OpenAI. *The Exponential Time Hypothesis is False.* Preprint, October 2026; revised October 25, 2026. Manuscript, main theorem. Accessed October 29, 2026.

[4] OpenAI. *The Exponential Time Hypothesis is False: Lean formalization scope.* 2026. Formalization scope note. Accessed October 29, 2026.

[5] Jacob A. White. On Multivariate Chromatic Polynomials of Hypergraphs and Hyperedge Elimination. *Electronic Journal of Combinatorics*, 18(1):P160, 2011. doi:10.37236/647.